

Scaling Language Model Inference via Gradient Estimation

Hongyang Ryan Zhang

Assistant Professor, Khoury College of Computer Sciences
Northeastern University, Boston

October 27, 2025

Motivation

Traditionally, computation relies on CPU: Central Processing Unit

Increasingly, a lot of the computation around LLMs are run on GPU: Graphics Processing Unit



LLM Inference: Given input x , a language model f_W parameterized by W , compute the output of the model $f_W(x)$. This routine is now used in many AI applications (e.g., speech recognition, self-driving cars, and image generation)

Motivation

Traditionally, computation relies on CPU: Central Processing Unit

Increasingly, a lot of the computation around LLMs are run on GPU: Graphics Processing Unit



LLM Inference: Given input x , a language model f_W parameterized by W , compute the output of the model $f_W(x)$. This routine is now used in many AI applications (e.g., speech recognition, self-driving cars, and image generation)

Motivation

Traditionally, computation relies on CPU: Central Processing Unit

Increasingly, a lot of the computation around LLMs are run on GPU: Graphics Processing Unit



LLM Inference: Given input x , a language model f_W parameterized by W , compute the output of the model $f_W(x)$. This routine is now used in many AI applications (e.g., speech recognition, self-driving cars, and image generation)

Motivation

Traditionally, computation relies on CPU: Central Processing Unit

Increasingly, a lot of the computation around LLMs are run on GPU: Graphics Processing Unit



LLM Inference: Given input x , a language model f_W parameterized by W , compute the output of the model $f_W(x)$. This routine is now used in many AI applications (e.g., speech recognition, self-driving cars, and image generation)



Motivation

Traditionally, computation relies on CPU: Central Processing Unit

Increasingly, a lot of the computation around LLMs are run on GPU: Graphics Processing Unit



LLM Inference: Given input x , a language model f_W parameterized by W , compute the output of the model $f_W(x)$. This routine is now used in many AI applications (e.g., speech recognition, self-driving cars, and image generation)

Precision	Bytes/param	Memory footprint (Llama/Qwen 8B)
FP32	4	32 GB
FP16	2	16 GB
Int8	1	8 GB
4-bit (Quantized)	0.5	4 GB

Table 1: Memory to store the weights of an 8B LLM by precision

Demonstration/Prompt selection

- LLM Chatbot: $\underbrace{\textit{Sequence of words}}_A \text{ (Prompt)} \rightarrow \underbrace{\textit{Next word}}_B \text{ (Answer)}$
- How do we select the prompt sequence to provide the best answer?
- This problem is related to **test-time compute**, which has been studied from both machine learning, and systems perspectives in recent work [DFE+22]

Demonstration/Prompt selection

- LLM Chatbot: $\underbrace{\textit{Sequence of words}}_A \text{ (Prompt)} \rightarrow \underbrace{\textit{Next word}}_B \text{ (Answer)}$
- How do we select the prompt sequence to provide the best answer?
- This problem is related to *test-time compute*, which has been studied from both machine learning, and systems perspectives in recent work [DFE+22]

Demonstration/Prompt selection

- LLM Chatbot: $\underbrace{\text{Sequence of words}}_A \text{ (Prompt)} \rightarrow \underbrace{\text{Next word}}_B \text{ (Answer)}$
- How do we select the prompt sequence to provide the best answer?
- This problem is related to **test-time compute**, which has been studied from both machine learning, and systems perspectives in recent work [DFE+22]

Demonstration/Prompt selection

- LLM Chatbot: $\underbrace{\text{Sequence of words}}_A \text{ (Prompt)} \rightarrow \underbrace{\text{Next word}}_B \text{ (Answer)}$
- How do we select the prompt sequence to provide the best answer?
- This problem is related to **test-time compute**, which has been studied from both machine learning, and systems perspectives in recent work [DFE+22]

This talk will be based on two references from my group

- *Linear-Time Demonstration Selection for In-Context Learning via Gradient Estimation* [ZZL+25]
- *Efficient Estimation of Kernel Surrogate Models for Task Attribution* [ZDZ25]

Problem setup

In-context learning: Given k examples and a query, $(x_1, y_1, \dots, x_k, y_k, x_{\text{query}})$, output y_{query} . Example: $\underbrace{(1 + 2, 3, 2 + 4, 6, \dots, 1 + 4, 5, 3 + 4)}_A \rightarrow \underbrace{7}_B$

- Training set: $\mathcal{S}^{\text{Train}} = \{(x_i^{\text{Train}}, y_i^{\text{Train}})\}_{i=1}^{n^{\text{Train}}}$
- Demonstration set: $\mathcal{S}^{\text{Demo}} = \{(x_i^{\text{Demo}}, y_i^{\text{Demo}})\}_{i=1}^{n^{\text{Demo}}}$

Problem setup

In-context learning: Given k examples and a query, $(x_1, y_1, \dots, x_k, y_k, x_{\text{query}})$, output y_{query} . Example: $\underbrace{(1 + 2, 3, 2 + 4, 6, \dots, 1 + 4, 5, 3 + 4)}_A \rightarrow \underbrace{7}_B$

- Training set: $\mathcal{S}^{\text{Train}} = \{(x_i^{\text{Train}}, y_i^{\text{Train}})\}_{i=1}^{n^{\text{Train}}}$
- Demonstration set: $\mathcal{S}^{\text{Demo}} = \{(x_i^{\text{Demo}}, y_i^{\text{Demo}})\}_{i=1}^{n^{\text{Demo}}}$

Problem setup

In-context learning: Given k examples and a query, $(x_1, y_1, \dots, x_k, y_k, x_{\text{query}})$, output y_{query} . Example: $\underbrace{(1 + 2, 3, 2 + 4, 6, \dots, 1 + 4, 5, 3 + 4)}_A \rightarrow \underbrace{7}_B$

- Training set: $\mathcal{S}^{\text{Train}} = \{(x_i^{\text{Train}}, y_i^{\text{Train}})\}_{i=1}^{n^{\text{Train}}}$
- Demonstration set: $\mathcal{S}^{\text{Demo}} = \{(x_i^{\text{Demo}}, y_i^{\text{Demo}})\}_{i=1}^{n^{\text{Demo}}}$

Problem setup

In-context learning: Given k examples and a query, $(x_1, y_1, \dots, x_k, y_k, x_{\text{query}})$, output y_{query} . Example: $\underbrace{(1 + 2, 3, 2 + 4, 6, \dots, 1 + 4, 5, 3 + 4)}_A \rightarrow \underbrace{7}_B$

- Training set: $\mathcal{S}^{\text{Train}} = \{(x_i^{\text{Train}}, y_i^{\text{Train}})\}_{i=1}^{n^{\text{Train}}}$
- Demonstration set: $\mathcal{S}^{\text{Demo}} = \{(x_i^{\text{Demo}}, y_i^{\text{Demo}})\}_{i=1}^{n^{\text{Demo}}}$

Objective function: given model f_W , embedding function ϕ , define

$$h(S) = \frac{1}{n^{\text{Train}}} \sum_{i=1}^{n^{\text{Train}}} \ell(f_W(\phi(S, x_i)), y_i)$$

Problem setup

In-context learning: Given k examples and a query, $(x_1, y_1, \dots, x_k, y_k, x_{\text{query}})$, output y_{query} . Example: $\underbrace{(1 + 2, 3, 2 + 4, 6, \dots, 1 + 4, 5, 3 + 4)}_A \rightarrow \underbrace{7}_B$

- Training set: $\mathcal{S}^{\text{Train}} = \{(x_i^{\text{Train}}, y_i^{\text{Train}})\}_{i=1}^{n^{\text{Train}}}$
- Demonstration set: $\mathcal{S}^{\text{Demo}} = \{(x_i^{\text{Demo}}, y_i^{\text{Demo}})\}_{i=1}^{n^{\text{Demo}}}$

Objective function: given model f_W , embedding function ϕ , define

$$h(S) = \frac{1}{n^{\text{Train}}} \sum_{i=1}^{n^{\text{Train}}} \ell(f_W(\phi(S, x_i)), y_i)$$

Problem statement: select a subset $S \subseteq \mathcal{S}^{\text{Demo}}$ to minimize $h(S)$

- Enumerate $S_1, \dots, S_m \subseteq \mathcal{S}^{\text{Demo}}$, compute $h(S_1), \dots, h(S_m)$
- This talk: an algorithm to quickly estimate $h(S_1), \dots, h(S_m)$ without having to run through full inference on all the subsets

Problem setup

In-context learning: Given k examples and a query, $(x_1, y_1, \dots, x_k, y_k, x_{\text{query}})$, output y_{query} . Example: $\underbrace{(1 + 2, 3, 2 + 4, 6, \dots, 1 + 4, 5, 3 + 4)}_A \rightarrow \underbrace{7}_B$

- Training set: $\mathcal{S}^{\text{Train}} = \{(x_i^{\text{Train}}, y_i^{\text{Train}})\}_{i=1}^{n^{\text{Train}}}$
- Demonstration set: $\mathcal{S}^{\text{Demo}} = \{(x_i^{\text{Demo}}, y_i^{\text{Demo}})\}_{i=1}^{n^{\text{Demo}}}$

Objective function: given model f_W , embedding function ϕ , define

$$h(S) = \frac{1}{n^{\text{Train}}} \sum_{i=1}^{n^{\text{Train}}} \ell(f_W(\phi(S, x_i)), y_i)$$

Problem statement: select a subset $S \subseteq \mathcal{S}^{\text{Demo}}$ to minimize $h(S)$

- Enumerate $S_1, \dots, S_m \subseteq \mathcal{S}^{\text{Demo}}$, compute $h(S_1), \dots, h(S_m)$
- This talk: **an algorithm to quickly estimate $h(S_1), \dots, h(S_m)$ without having to run through full inference on all the subsets**

Key observation: First-order approximation in the embedding space

Recall that ϕ is an embedding function. Given a prompt subset $S \subseteq \mathcal{S}^{\text{Train}}$ on an input x , denoted as $f_W(\phi(S, x))$, the first-order approximation of f_W around the embedding vector $\phi(S_0, x)$ is given by:

$$f_W(\phi(S, x)) = f_W(\phi(S_0, x)) + \underbrace{\langle \nabla_{\phi} f_W(\phi(S_0, x)), \phi(S, x) - \phi(S_0, x) \rangle}_{\text{Input gradient Distance}} + \epsilon_{S,x}$$

- ∇_{ϕ} : gradient of f_W with respect to the input embedding ϕ
- $\epsilon_{S,x}$: approximation error

Key observation: First-order approximation in the embedding space

Recall that ϕ is an embedding function. Given a prompt subset $S \subseteq \mathcal{S}^{\text{Train}}$ on an input x , denoted as $f_W(\phi(S, x))$, the first-order approximation of f_W around the embedding vector $\phi(S_0, x)$ is given by:

$$f_W(\phi(S, x)) = f_W(\phi(S_0, x)) + \underbrace{\langle \nabla_{\phi} f_W(\phi(S_0, x)), \phi(S, x) - \phi(S_0, x) \rangle}_{\text{Input gradient Distance}} + \epsilon_{S,x}$$

- ∇_{ϕ} : gradient of f_W with respect to the input embedding ϕ
- $\epsilon_{S,x}$: approximation error

Key observation: First-order approximation in the embedding space

Recall that ϕ is an embedding function. Given a prompt subset $S \subseteq \mathcal{S}^{\text{Train}}$ on an input x , denoted as $f_W(\phi(S, x))$, the first-order approximation of f_W around the embedding vector $\phi(S_0, x)$ is given by:

$$f_W(\phi(S, x)) = f_W(\phi(S_0, x)) + \underbrace{\langle \nabla_{\phi} f_W(\phi(S_0, x)), \phi(S, x) - \phi(S_0, x) \rangle}_{\text{Input gradient}} + \underbrace{\epsilon_{S,x}}_{\text{Distance}}$$

- ∇_{ϕ} : gradient of f_W with respect to the input embedding ϕ
- $\epsilon_{S,x}$: approximation error

Table 2: Relative approximation error of $\epsilon_{S,x}$, tested on LLMs (of up to 34 billion parameters) with a sentiment recognition dataset

Distance	DeepSeek-7B	Llama-13B	CodeLlama-34B
15% – 20%	0.21 $\pm$ 0.04%	0.06 $\pm$ 0.00%	0.06 $\pm$ 0.01%
20% – 25%	0.30 $\pm$ 0.05%	0.08 $\pm$ 0.02%	0.06 $\pm$ 0.01%
25% – 30%	0.51 $\pm$ 0.04%	0.09 $\pm$ 0.01%	0.08 $\pm$ 0.02%
30% – 35%	0.53 $\pm$ 0.07%	0.28 $\pm$ 0.03%	0.13 $\pm$ 0.03%
35% – 40%	1.40 $\pm$ 0.20%	0.37 $\pm$ 0.03%	0.30 $\pm$ 0.03%

A case study of linear functions

Case study: in-context learning of linear functions [GTL+22]

- In-context examples $(x_1, y_1), (x_2, y_2), \dots, (x_k, y_k)$, where $y_i = \langle \beta^{(j)}, x_i \rangle + \epsilon_i$
- Coefficient vector $\beta^{(j)} \in \mathbb{R}^{20}$ is randomly drawn from a distribution of β

A case study of linear functions

Case study: in-context learning of linear functions [GTL+22]

- In-context examples $(x_1, y_1), (x_2, y_2), \dots, (x_k, y_k)$, where $y_i = \langle \beta^{(i)}, x_i \rangle + \epsilon_i$
- Coefficient vector $\beta^{(i)} \in \mathbb{R}^{20}$ is randomly drawn from a distribution of β

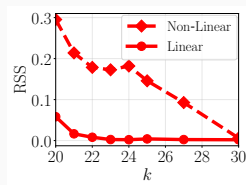


Figure 2: Showing low approximation error relative to full inference as k grows

A case study of linear functions

Case study: in-context learning of linear functions [GTL+22]

- In-context examples $(x_1, y_1), (x_2, y_2), \dots, (x_k, y_k)$, where $y_i = \langle \beta^{(j)}, x_i \rangle + \epsilon_i$
- Coefficient vector $\beta^{(j)} \in \mathbb{R}^{20}$ is randomly drawn from a distribution of β

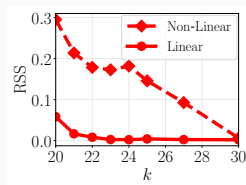


Figure 2: Showing low approximation error relative to full inference as k grows

Embedding similarity methods (top- k , random- k) fail to capture the input-output mappings

Estimating relevance scores using a random ensemble

Algorithm

1. $S_1, \dots, S_m \leftarrow$ Sample m random subsets each of size k from $\mathcal{S}^{\text{Demo}}$
2. $A^{(1)}, \dots, A^{(\alpha)} \leftarrow$ Randomly choose α anchor subsets from $S_1, S_2, \dots, S_m$
3. $\hat{h}^{(i)}(S_j) \leftarrow$ estimate $h(A^{(i)}, S_j)$ for every $i = 1, \dots, \alpha$ and $j = 1, \dots, m$
Then for $q = 1, 2, \dots, n^{\text{Demo}}$,

$$s_q \leftarrow \frac{1}{\alpha} \sum_{i=1}^{\alpha} \text{Avg}(\{\hat{h}^{(i)}(S_j) \mid (x_q^{\text{Demo}}, y_q^{\text{Demo}}) \in S_j\}) \quad (1)$$

4. $S_k \subseteq \mathcal{S}^{\text{Demo}} \leftarrow k$ demonstrations with the lowest s_q scores

Estimating relevance scores using a random ensemble

Algorithm

1. $S_1, \dots, S_m \leftarrow$ Sample m random subsets each of size k from $\mathcal{S}^{\text{Demo}}$
2. $A^{(1)}, \dots, A^{(\alpha)} \leftarrow$ Randomly choose α anchor subsets from $S_1, S_2, \dots, S_m$
3. $\hat{h}^{(i)}(S_j) \leftarrow$ estimate $h(A^{(i)}, S_j)$ for every $i = 1, \dots, \alpha$ and $j = 1, \dots, m$
Then for $q = 1, 2, \dots, n^{\text{Demo}}$,

$$s_q \leftarrow \frac{1}{\alpha} \sum_{i=1}^{\alpha} \text{Avg}(\{\hat{h}^{(i)}(S_j) \mid (x_q^{\text{Demo}}, y_q^{\text{Demo}}) \in S_j\}) \quad (1)$$

4. $S_k \subseteq \mathcal{S}^{\text{Demo}} \leftarrow k$ demonstrations with the lowest s_q scores

Estimating relevance scores using a random ensemble

Algorithm

1. $S_1, \dots, S_m \leftarrow$ Sample m random subsets each of size k from $\mathcal{S}^{\text{Demo}}$
2. $A^{(1)}, \dots, A^{(\alpha)} \leftarrow$ Randomly choose α anchor subsets from $S_1, S_2, \dots, S_m$
3. $\hat{h}^{(i)}(S_j) \leftarrow$ estimate $h(A^{(i)}, S_j)$ for every $i = 1, \dots, \alpha$ and $j = 1, \dots, m$
Then for $q = 1, 2, \dots, n^{\text{Demo}}$,

$$s_q \leftarrow \frac{1}{\alpha} \sum_{i=1}^{\alpha} \text{Avg}(\{\hat{h}^{(i)}(S_j) \mid (x_q^{\text{Demo}}, y_q^{\text{Demo}}) \in S_j\}) \quad (1)$$

4. $S_k \subseteq \mathcal{S}^{\text{Demo}} \leftarrow k$ demonstrations with the lowest s_q scores

Estimating relevance scores using a random ensemble

Algorithm

1. $S_1, \dots, S_m \leftarrow$ Sample m random subsets each of size k from $\mathcal{S}^{\text{Demo}}$
2. $A^{(1)}, \dots, A^{(\alpha)} \leftarrow$ Randomly choose α anchor subsets from $S_1, S_2, \dots, S_m$
3. $\hat{h}^{(i)}(S_j) \leftarrow$ estimate $h(A^{(i)}, S_j)$ for every $i = 1, \dots, \alpha$ and $j = 1, \dots, m$
Then for $q = 1, 2, \dots, n^{\text{Demo}}$,

$$s_q \leftarrow \frac{1}{\alpha} \sum_{i=1}^{\alpha} \text{Avg}(\{\hat{h}^{(i)}(S_j) \mid (x_q^{\text{Demo}}, y_q^{\text{Demo}}) \in S_j\}) \quad (1)$$

4. $S_k \subseteq \mathcal{S}^{\text{Demo}} \leftarrow k$ demonstrations with the lowest s_q scores

Estimating relevance scores using a random ensemble

Algorithm

1. $S_1, \dots, S_m \leftarrow$ Sample m random subsets each of size k from $\mathcal{S}^{\text{Demo}}$
2. $A^{(1)}, \dots, A^{(\alpha)} \leftarrow$ Randomly choose α anchor subsets from $S_1, S_2, \dots, S_m$
3. $\hat{h}^{(i)}(S_j) \leftarrow$ estimate $h(A^{(i)}, S_j)$ for every $i = 1, \dots, \alpha$ and $j = 1, \dots, m$
Then for $q = 1, 2, \dots, n^{\text{Demo}}$,

$$s_q \leftarrow \frac{1}{\alpha} \sum_{i=1}^{\alpha} \text{Avg}(\{\hat{h}^{(i)}(S_j) \mid (x_q^{\text{Demo}}, y_q^{\text{Demo}}) \in S_j\}) \quad (1)$$

4. $S_k \subseteq \mathcal{S}^{\text{Demo}} \leftarrow k$ demonstrations with the lowest s_q scores

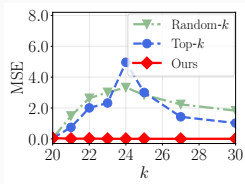


Figure 3: Showing that the selected demonstrations achieves lower error than embedding similarity methods (top- k / random- k) in linear regression

Estimating relevance scores using a random ensemble

Algorithm

1. $S_1, \dots, S_m \leftarrow$ Sample m random subsets each of size k from $\mathcal{S}^{\text{Demo}}$
2. $A^{(1)}, \dots, A^{(\alpha)} \leftarrow$ Randomly choose α anchor subsets from $S_1, S_2, \dots, S_m$
3. $\hat{h}^{(i)}(S_j) \leftarrow$ estimate $h(A^{(i)}, S_j)$ for every $i = 1, \dots, \alpha$ and $j = 1, \dots, m$
Then for $q = 1, 2, \dots, n^{\text{Demo}}$,

$$s_q \leftarrow \frac{1}{\alpha} \sum_{i=1}^{\alpha} \text{Avg}(\{\hat{h}^{(i)}(S_j) \mid (x_q^{\text{Demo}}, y_q^{\text{Demo}}) \in S_j\}) \quad (1)$$

4. $S_k \subseteq \mathcal{S}^{\text{Demo}} \leftarrow k$ demonstrations with the lowest s_q scores

Running time: compute model outputs with one forward pass of time T and input gradients with time $2T$, plus an overhead cost of $O(n \log n)$, which is negligible relative to T

- T : time cost of one forward pass on f_W with training set
- k : number of the selected demonstrations
- n : total size of the demonstration set

Summary of empirical results

Estimation results:

Math reasoning	DeepSeek-7B	CodeLlama-34B	Speedup
Estimation error	$0.08 \pm 0.1\%$	$0.07 \pm 0.0\%$	19.7×
Sentiment recognition	DeepSeek-7B	CodeLlama-34B	Speedup
Estimation error	$0.18 \pm 0.0\%$	$0.36 \pm 0.0\%$	10.8×

Summary of empirical results

Estimation results:

Math reasoning	DeepSeek-7B	CodeLlama-34B	Speedup
Estimation error	$0.08 \pm 0.1\%$	$0.07 \pm 0.0\%$	19.7×
Sentiment recognition	DeepSeek-7B	CodeLlama-34B	Speedup
Estimation error	$0.18 \pm 0.0\%$	$0.36 \pm 0.0\%$	10.8×

Selection results: loss-based selection outperforms embedding similarity methods by 11%, with 49% less computation (averaged over six sentiment recognition and math reasoning tasks)

Summary of empirical results

Estimation results:

Math reasoning	DeepSeek-7B	CodeLlama-34B	Speedup
Estimation error	$0.08_{\pm 0.1}\%$	$0.07_{\pm 0.0}\%$	19.7×
Sentiment recognition	DeepSeek-7B	CodeLlama-34B	Speedup
Estimation error	$0.18_{\pm 0.0}\%$	$0.36_{\pm 0.0}\%$	10.8×

Long-context evaluations: $k = 150 \approx 9000$ tokens

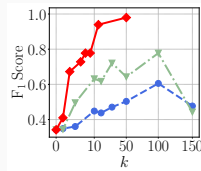
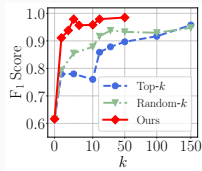


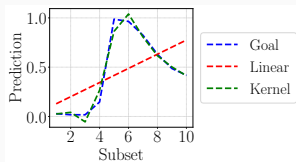
Figure 3: Comparing our estimation with top- k and random- k by varying k on sentiment recognition and math reasoning tasks using DeepSeek-7B

Extension: Nonlinear surrogate models

Our algorithm assumes an additive score function (equation (1)), and does not model nonlinear task interactions such as synergy, antagonism, or XOR-type relationships

Extension: Nonlinear surrogate models

Our algorithm assumes an additive score function (equation (1)), and does not model nonlinear task interactions such as synergy, antagonism, or XOR-type relationships

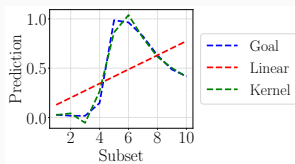


Residual error	Linear	RBF
CIFAR-10	4.4 ± 0.9	1.0 ± 0.0
Modular arithmetic	4.6 ± 1.3	1.5 ± 0.4
In-context learning	0.8 ± 0.2	0.4 ± 0.1
Multi-objective RL	0.2 ± 0.1	0.1 ± 0.1

Figure 4: Illustrate linear vs. kernel models on a decision boundary. Using the RBF kernel leads to much better fit than linear surrogate models.

Extension: Nonlinear surrogate models

Our algorithm assumes an additive score function (equation (1)), and does not model nonlinear task interactions such as synergy, antagonism, or XOR-type relationships



Residual error	Linear	RBF
CIFAR-10	4.4 ± 0.9	1.0 ± 0.0
Modular arithmetic	4.6 ± 1.3	1.5 ± 0.4
In-context learning	0.8 ± 0.2	0.4 ± 0.1
Multi-objective RL	0.2 ± 0.1	0.1 ± 0.1

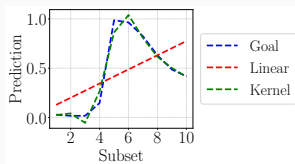
Figure 4: Illustrate linear vs. kernel models on a decision boundary. Using the RBF kernel leads to much better fit than linear surrogate models.

Using gradient estimation for estimating kernel ridge regression [ZDZ25]

- Pre-compute gradients and functional values only once
- Use cases in transformers, in-context learning, and multi-objective reinforcement learning

Extension: Nonlinear surrogate models

Our algorithm assumes an additive score function (equation (1)), and does not model nonlinear task interactions such as synergy, antagonism, or XOR-type relationships



Residual error	Linear	RBF
CIFAR-10	4.4 ± 0.9	1.0 ± 0.0
Modular arithmetic	4.6 ± 1.3	1.5 ± 0.4
In-context learning	0.8 ± 0.2	0.4 ± 0.1
Multi-objective RL	0.2 ± 0.1	0.1 ± 0.1

Figure 4: Illustrate linear vs. kernel models on a decision boundary. Using the RBF kernel leads to much better fit than linear surrogate models.

Using gradient estimation for estimating kernel ridge regression [ZDZ25]

- Pre-compute gradients and functional values only once
- Use cases in **transformers, in-context learning, and multi-objective reinforcement learning**

Conclusion

In this work, we measure the loss surface of LLMs and use its geometry properties to design faster algorithms. **Key idea:**

- Pre-compute the gradients in the input embedding space once, and use gradients to estimate model outputs
- Use kernel surrogates to capture nonlinear dynamics

Open questions

- Supervised fine-tuning / alignment of multi-modal LLMs; incorporating ranking data
- Better understand the geometry of loss surfaces in language models, such as the Hessian spectrum of loss surfaces

Thank you for listening!

Conclusion

In this work, we measure the loss surface of LLMs and use its geometry properties to design faster algorithms. **Key idea:**

- Pre-compute the gradients in the input embedding space once, and use gradients to estimate model outputs
- Use kernel surrogates to capture nonlinear dynamics

Open questions

- Supervised fine-tuning / alignment of multi-modal LLMs; incorporating ranking data
- Better understand the geometry of loss surfaces in language models, such as the Hessian spectrum of loss surfaces

Thank you for listening!

Conclusion

In this work, we measure the loss surface of LLMs and use its geometry properties to design faster algorithms. **Key idea:**

- Pre-compute the gradients in the input embedding space once, and use gradients to estimate model outputs
- Use kernel surrogates to capture nonlinear dynamics

Open questions

- Supervised fine-tuning / alignment of multi-modal LLMs; incorporating ranking data
- Better understand the geometry of loss surfaces in language models, such as the Hessian spectrum of loss surfaces

Thank you for listening!

Conclusion

In this work, we measure the loss surface of LLMs and use its geometry properties to design faster algorithms. **Key idea:**

- Pre-compute the gradients in the input embedding space once, and use gradients to estimate model outputs
- Use kernel surrogates to capture nonlinear dynamics

Open questions

- Supervised fine-tuning / alignment of multi-modal LLMs; incorporating ranking data
- Better understand the geometry of loss surfaces in language models, such as the Hessian spectrum of loss surfaces

Thank you for listening!

Acknowledgment



Ziniu Zhang



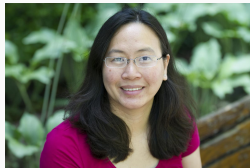
Zhenshuo Zhang



Dongyue Li



Lu Wang (Associate Prof, Michigan)

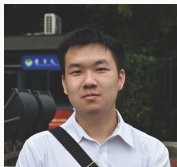


Jennifer Dy (Professor, Northeastern)

Acknowledgment



Ziniu Zhang



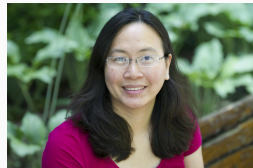
Zhenshuo Zhang



Dongyue Li



Lu Wang (Associate Prof, Michigan)



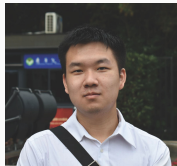
Jennifer Dy (Professor, Northeastern)

Funding source: NSF IIS-2412008, JP Morgan Chase (through a PhD fellowship fund)

Acknowledgment



Ziniu Zhang



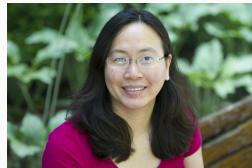
Zhenshuo Zhang



Dongyue Li



Lu Wang (Associate Prof, Michigan)



Jennifer Dy (Professor, Northeastern)

Funding source: NSF IIS-2412008, JP Morgan Chase (through a PhD fellowship fund)

Lab page: <https://virtuosoresearch.github.io/>

-  Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. (2022). **“Flashattention: Fast and memory-efficient exact attention with io-awareness”**. In: *Advances in neural information processing systems* 35, pp. 16344–16359 (7–10).
-  Garg, S., Tsipras, D., Liang, P. S., and Valiant, G. (2022). **“What can transformers learn in-context? a case study of simple function classes”**. In: *Advances in Neural Information Processing Systems (NeurIPS)* 35, pp. 30583–30598 (20–22).
-  Zhang, Z., Duan, M., and Zhang, H. R. (2025a). **“Efficient Estimation of Kernel Surrogate Models for Task Attribution”**. In: *Under Review* (10, 34, 35).
-  Zhang, Z., Zhang, Z., Li, D., Wang, L., Dy, J., and Zhang, H. R. (2025b). **“Linear-Time Demonstration Selection for In-Context Learning via Gradient Estimation”**. In: *EMNLP* (10).